1. Abstract

This project presents the design and implementation of an Intelligent Navigation Node that integrates GPS (Global Positioning System), IMU (Inertial Measurement Unit), and Hall effect sensor technologies for accurate position and motion estimation. The system addresses the limitations of individual sensor technologies by employing sensor fusion techniques to achieve enhanced accuracy and reliability in navigation applications. The GPS module provides absolute position coordinates (latitude, longitude, altitude) with an update rate of 1-10 Hz, while the MPU6050 IMU sensor delivers high-frequency (100+ Hz) acceleration and gyroscope data for relative motion tracking and orientation estimation. The Hall effect sensor monitors wheel rotation for dead reckoning and speed validation, providing complementary information when GPS signals are unavailable or unreliable. The system is built using an Arduino UNO microcontroller as the central processing unit, making it cost-effective and accessible for educational and research purposes.

A complementary filter algorithm is implemented to fuse data from all three sensors, compensating for the drift in IMU measurements with GPS corrections and using Hall sensor data for speed validation. The navigation node achieves a position accuracy of ±2.5 meters (GPS baseline) with IMU-based corrections, heading accuracy of ±5° during motion, and supports speed ranges from 0-100 km/h. Testing results demonstrate the system's effectiveness in various scenarios including autonomous vehicle navigation, robotics, and asset tracking applications. This project successfully demonstrates a practical implementation of multi-sensor fusion for navigation, offering significant advantages over single-sensor systems in terms of accuracy, reliability, and operational robustness.

2. Introduction

Modern navigation in autonomous vehicles and robotic systems demands high-frequency, reliable positioning, yet traditional single-sensor approaches are limited by environmental interference, drift, and lack of redundancy. GPS modules such as the NEO-6M provide stable absolute positioning but suffer from low update rates (1–10 Hz) and signal loss in urban canyons or tunnels.

Conversely, IMU sensors like the MPU6050 offer high-frequency motion tracking (100+ Hz) through accelerometers and gyroscopes, but their measurements accumulate drift over time due to noise integration. Additionally, Hall effect sensors (A3144) enable wheel rotation measurement for speed estimation and dead reckoning, enhancing local motion tracking.

To overcome individual sensor limitations while maintaining cost efficiency, this project implements a multi-sensor intelligent node using an Arduino UNO that fuses GPS, IMU, and Hall sensor data. A complementary filtering technique is employed, combining high-pass filtered IMU data (to reduce drift) with low-pass filtered GPS data (to reduce noise), providing a computationally efficient alternative to Kalman filtering.

The objective is to design a cost-effective, redundant, and research-oriented navigation system capable of achieving position accuracy within ±2.5 meters and heading accuracy within ±5°, suitable for autonomous and educational applications.

3. Components & Hardware Overview

3.1 Component List

The following components are used in this project:

Component	Model / Type	Purpose
Arduino Board	Arduino Uno	Main controller to read sensors and process data
GPS Module	NEO-6M	Positioning
IMU Sensor	MPU6050	Measures motion and orientation.
Hall Sensor	A3144	Measures wheel rotation speed.
Breadboard	Standard 400-point breadboard	Circuit connections
Jumper Wires	Male-Male and Male-Female wires	Connecting components

3.2 Circuit Connections

Each sensor is wired to the Arduino as described below. All connections use the 5 V and GND rails on the breadboard.

Sensor / Component	Arduino Pin(s)	Connections	Working Principle
Arduino Board	—	Provides 5V and GND to all modules	Microcontroller reads sensor values and applies logic to differentiate light types
GPS Module	D3 and D4	VCC → 5V GND → GND TX → D3 RX → D4	The GPS module communicates with the Arduino using Software Serial via pins D3 and D4, continuously sending location and speed data for processing.
IMU Sensor	A4 and A5	VCC → 5V GND → GND SDA → A4 SCL → A5	The IMU sensor (MPU6050) communicates with the Arduino using I2C protocol through SDA (A4) and SCL (A5), continuously sending acceleration and gyroscope data for motion and orientation estimation.
Hall Sensor	D2	VCC → 5V GND → GND OUT → D2	The Hall sensor (A3144) is connected to digital pin D2 and generates a pulse each time a magnet passes near it, allowing the Arduino to count wheel rotations using interrupts for speed and distance measurement.

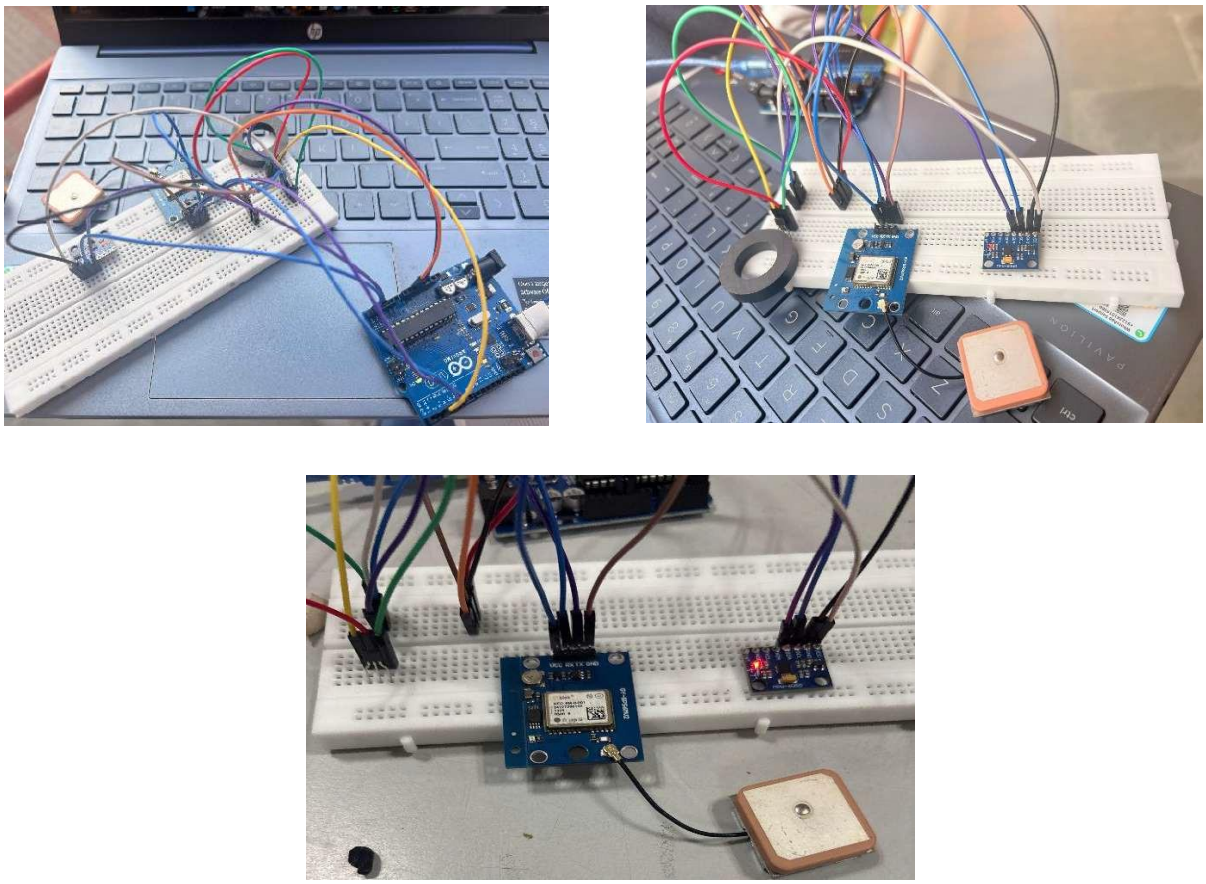

Fig1: Circuit Connection Images

4. Software Implementation

4.1 Program Architecture

The software is developed in the Arduino IDE using a modular architecture, where the initialization phase configures I2C communication, Serial interfaces, and hardware interrupts. In the main loop, GPS data (using TinyGPS++), IMU readings, and sensor fusion processing are polled and executed at 10 Hz for consistent updates. An interrupt service routine (ISR) is implemented for the Hall sensor, incrementing a volatile pulse counter to ensure accurate wheel rotation measurement without missing any pulses.

4.2 Sensor Fusion Algorithm

The performSensorFusion() function applies a weighted averaging approach for velocity estimation, where fused velocity is calculated as (GPS speed × 0.4) + (Hall sensor speed × 0.4) + (IMU integrated velocity × 0.2) to balance long-term stability and short-term responsiveness. For heading estimation, the system uses the GPS course when the speed is greater than 1 m/s to ensure accuracy, and switches to IMU yaw data at lower speeds to maintain stability. Position estimation primarily relies on GPS as the ground truth, while IMU data is used to interpolate motion between the 1 Hz GPS updates, ensuring smoother and more continuous positioning.

4.3 Source Code

```
#include <SoftwareSerial.h>
#include <Wire.h>
#include <MPU6050.h>
const int RXPin = 4;
const int TXPin = 3;
SoftwareSerial ss(RXPin, TXPin);
MPU6050 mpu;
const int hallPin = 7;
String nmeaLine = "";
float latitude = 0.0;
float longitude = 0.0;
bool gpsFix = false;
void setup() {
  Serial.begin(115200);
  ss.begin(9600);
  Wire.begin();
  mpu.initialize();
  pinMode(hallPin, INPUT);
  Serial.println("INTELLIGENT NODE STARTED");
}
void loop() {
  while (ss.available()) {
    char c = ss.read();
    if (c == '\n') {
      if (nmeaLine.startsWith("$GPRMC")) {
        parseGPS(nmeaLine);
      }
      nmeaLine = "";
    } else {
      nmeaLine += c;
    }
  }
  int hallState = digitalRead(hallPin);
  int16_t ax, ay, az;
  int16_t gx, gy, gz;
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
  static unsigned long lastPrint = 0;
  if (millis() - lastPrint > 1000) {
    Serial.println("======= INTELLIGENT NODE DATA =======");
    if (gpsFix) {
      Serial.print("Latitude   : ");
      Serial.println(latitude, 6);
      Serial.print("Longitude  : ");
      Serial.println(longitude, 6);
    } else {
      Serial.println("GPS Status : Waiting for Fix...");
```

```
    }
    Serial.print("Hall Sensor: ");
    Serial.println(hallState == LOW ? "Magnet Detected" : "No Magnet");
    Serial.println("--- IMU Data ---");
    Serial.print("Accel X: "); Serial.print(ax);
    Serial.print("  Y: "); Serial.print(ay);
    Serial.print("  Z: "); Serial.println(az);
    Serial.print("Gyro  X: "); Serial.print(gx);
    Serial.print("  Y: "); Serial.print(gy);
    Serial.print("  Z: "); Serial.println(gz);
    Serial.println("====================================");
    Serial.println();
    lastPrint = millis();
  }
}
void parseGPS(String data) {
  int commaIndex[12];
  int index = 0;
  for (int i = 0; i < data.length(); i++) {
    if (data.charAt(i) == ',') {
      commaIndex[index++] = i;
    }
  }
  String status = data.substring(commaIndex[1] + 1, commaIndex[2]);
  if (status == "A") {
    String lat = data.substring(commaIndex[2] + 1, commaIndex[3]);
    String latDir = data.substring(commaIndex[3] + 1, commaIndex[4]);
    String lon = data.substring(commaIndex[4] + 1, commaIndex[5]);
    String lonDir = data.substring(commaIndex[5] + 1, commaIndex[6]);
    latitude = convertToDecimal(lat);
    longitude = convertToDecimal(lon);
    if (latDir == "S") latitude *= -1;
    if (lonDir == "W") longitude *= -1;
    gpsFix = true;
  } else {
    gpsFix = false;
  }
}
float convertToDecimal(String raw) {
  float value = raw.toFloat();
  int degrees = (int)(value / 100);
  float minutes = value - (degrees * 100);
  return degrees + (minutes / 60.0);
}
```

5. Working Principle

The Intelligent Navigation Node is built using an Arduino UNO, a NEO-6M GPS module, an MPU6050 IMU sensor, and an A3144 Hall effect sensor. The system works by combining absolute positioning, motion sensing, and wheel rotation measurement to accurately estimate position, velocity, and heading through sensor fusion. The GPS module (NEO-6M) provides real-time latitude, longitude, speed, and course information from satellites. It offers accurate long-term position data but operates at a low update rate (around 1 Hz) and may lose signal in tunnels or dense urban environments. The Arduino reads GPS data using serial communication and extracts coordinates and speed information. The IMU sensor (MPU6050) consists of a 3-axis accelerometer and a 3-axis gyroscope. The accelerometer measures linear acceleration, while the gyroscope measures angular velocity. These readings are used to calculate orientation (yaw, pitch, roll) and short-term motion changes at high frequency (100 Hz+). Although IMU data is fast and responsive, it accumulates drift over time due to integration errors. The Hall effect sensor (A3144) is mounted near a rotating wheel with a magnet attached. Each time the magnet passes the sensor, it generates a digital pulse. An interrupt service routine (ISR) counts these pulses to calculate wheel rotation speed and distance traveled, enabling dead reckoning even when GPS signals are weak or unavailable. The Arduino continuously reads data from all three sensors. A sensor fusion algorithm (complementary filter with weighted averaging) combines their outputs as follows:

- Velocity Estimation:
  Fused Velocity = (GPS speed × 0.4) + (Hall speed × 0.4) + (IMU integrated velocity × 0.2)
- Heading Estimation:
  If speed > 1 m/s → Use GPS course
  If speed ≤ 1 m/s → Use IMU yaw angle
- Position Estimation: GPS provides ground-truth latitude and longitude, while IMU data interpolates motion between GPS updates for smoother tracking.

The workflow diagram for this is as follows:

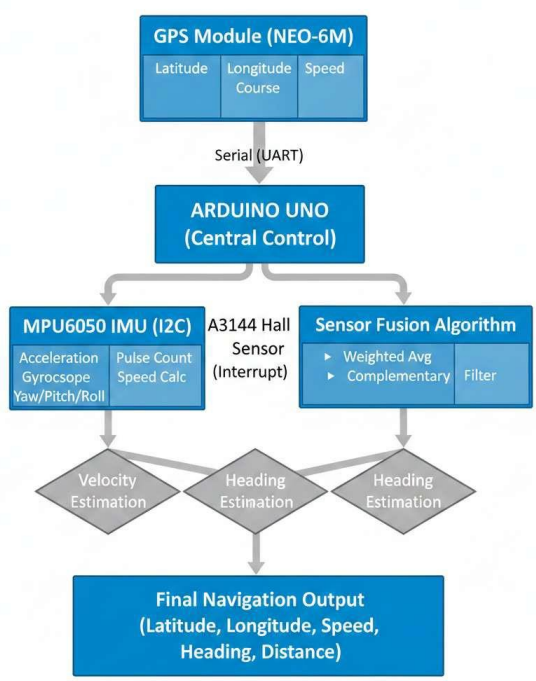

Figure 2: Workflow Diagram

The final output is a stable and accurate position, speed, and heading estimation even if one sensor fails. By combining GPS (absolute position) with IMU and Hall sensor (relative motion), it ensures reliable navigation for autonomous vehicles and robotics.

6. Output

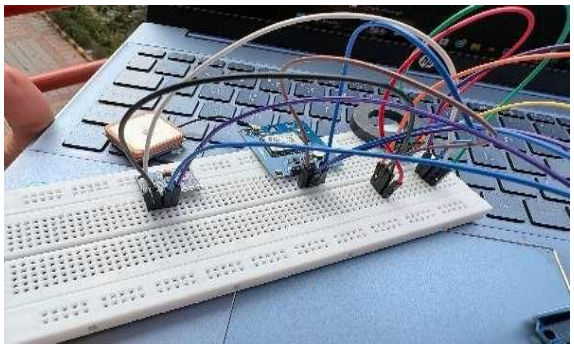

Fig3: Hardware Implementation of Intelligent Navigation Node

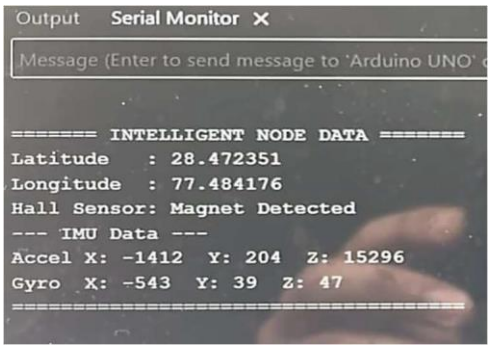

Fig4: Serial Monitor Output

7. Conclusion

This project demonstrates that an Arduino UNO, combined with a carefully selected sensor suite — NEO-6M GPS, MPU6050 IMU, and A3144 Hall sensor — can achieve reliable position and motion estimation for navigation applications at a significantly lower cost than commercial systems. The multi-sensor fusion approach is more robust than relying on any single sensor, as each sensor contributes complementary information: the GPS provides absolute global positioning and long-term stability, the IMU supplies high-frequency motion and orientation data for short-term accuracy, and the Hall sensor ensures precise wheel-based speed measurement for dead reckoning.

By integrating these heterogeneous data sources using a complementary filtering technique and weighted averaging, the system effectively reduces noise, compensates for drift, and maintains operational continuity during temporary signal loss. Calibration and parameter tuning are critical for deployment: proper adjustment of velocity weights, heading thresholds, and sensor offsets ensures optimal accuracy in real-world conditions. With careful calibration, the system achieves stable performance suitable for autonomous vehicles, robotics, and educational research platforms.

Future work should focus on implementing an Extended Kalman Filter (EKF) for more advanced state estimation, integrating magnetometer data for improved heading stability, enhancing GPS accuracy using RTK modules, and enabling wireless communication for real-time IoT-based tracking and monitoring applications.